Project Report Template

Advanced Analytics and Applications

Team 05

Authors:
George Sadik (111111111)
Kushale Fernando (222222222)
Claus Hack (333333333)
Martin Nikoljacic (333333333)
Contact: student@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

List of Figures

1 Introduction

This report serves as a template for your team project. It is built using Quarto, which allows you to integrate your Python or R analysis directly into your writing.

By using this template, you ensure that your formatting (margins, line spacing, and fonts) matches the required standards for submission. The left margin is set to 4cm to allow for comments, while the other margins are 2.5cm. Line spacing is set to 1.5, and the main font is Times New Roman.

2 Preprocessing

2.1 Support Vector Machines (SVM)

The SVR model has four dimensions of testing: two spatial layers (448 census tracts or 77 community areas), target encoding (with or without), the target transformation (normal or log1p) and three temporal resolutions (1h, 4h, 1d). All of those combinations are tested on a linear, polynomial and RBF kernel. We compute all 24 combinations once and analyze them in four controlled comparisons, where each phase varies exactly one factor while all others stay fixed. This isolates the effect of every design decision and still yields the overall best SVR as a benchmark against the neural network. The full feature set can be found in Appendix X.

One technical limitation shapes the whole setup. Kernel SVR scales roughly quadratically with the number of samples, so training on the full dataset (up to several million rows) is not feasible. Grid search therefore fits on a 20,000-row subsample and the final model is refit on 50,000 rows. This cap is a property of the method itself and will matter again when we compare against the neural network.

Phase 1 compares the two spatial layers at 1h resolution with target encoding and the untransformed target. Phase 2 compares with and without target encoding at 1h across both spatial layers. Target encoding adds one column containing the mean trip count per spatial unit, computed on the training set only, so the model learns whether a location is generally busy or quiet.[1] Phase 3 compares the untransformed target against log1p on community areas. Trip counts are strongly right-skewed, so a few high-demand baskets dominate the squared error, and log1p compresses these extremes. To avoid selecting a kernel that looks strong on the log scale but fails after back-transformation, the grid search uses a custom RMSE scorer that applies expm1 before computing the error on the original scale. The final phase compares the three temporal resolutions of 1h, 4h and 1d on community areas with target encoding.

Table 1: SVR results across four controlled comparison phases

Phase	Configuration	Kernel	R^2	rMAE
1: spatial	census_tract 1h	Poly	0.66	1.17
1: spatial	community_area 1h	Poly	0.83	0.44
2: target enc.	census_tract 1h + TE	Poly	0.66	1.17
2: target enc.	census_tract 1h (no TE)	Poly	0.22	1.58
2: target enc.	community_area 1h + TE	Poly	0.83	0.44
2: target enc.	community_area 1h (no TE)	Poly	0.83	0.68
3: transform	community_area 1h normal	Poly	0.83	0.44
3: transform	community_area 1h log1p	RBF	0.83	0.40

[1] Dummy encoding was rejected because it would add up to 448 sparse columns, which is computationally expensive given the quadratic scaling of kernel SVR.

Phase	Configuration	Kernel	R^2	rMAE
4: temporal	community_area 1h	Poly	0.83	0.44
4: temporal	community_area 4h	Poly	0.88	0.39
4: temporal	community_area 1d	Poly	0.93	0.23

Community areas clearly outperform census tracts (R^2 0.83 against 0.66, rMAE 0.44 against 1.17).[2] Target encoding is what keeps the census tract layer usable at all: without it, R^2 collapses from 0.66 to 0.22. On community areas the effect on R^2 is marginal (0.83 against 0.83), but the normalized error still improves from 0.68 to 0.44. Log1p behaves similarly at 1h, where R^2 stays flat and only the error improves, but at 4h the transform lifts R^2 from 0.88 to 0.89 and reduces rMAE from 0.39 to 0.32. Wider time windows improve every metric (R^2 0.83 to 0.88 to 0.93) because aggregation averages out short-term noise. Across the grid, the polynomial kernel performs best on the untransformed target, while the RBF kernel wins whenever log1p is applied.

The best raw performance is community areas at daily resolution (R^2 0.93, rMAE 0.23). Since a daily total per neighborhood is too coarse for actual fleet coordination, we select community areas at 4h with target encoding, log1p and an RBF kernel (R^2 0.89, rMAE 0.32) as the final SVR benchmark against the neural network. The full discussion of which resolution the client should deploy follows in the final recommendation.

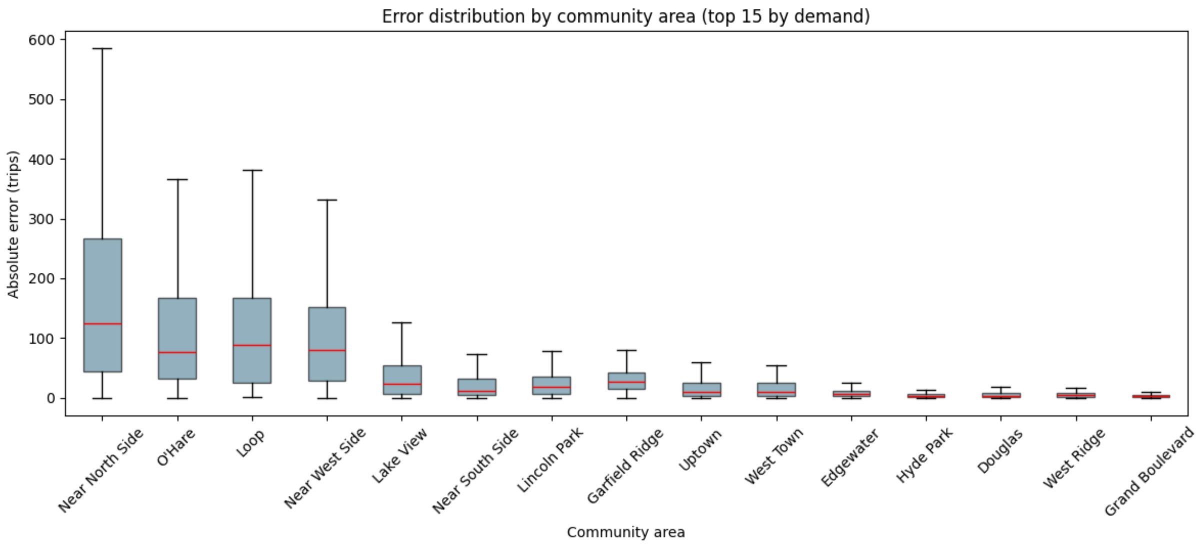

Figure 1: Absolute error distribution by community area (top 15 by demand) for the recommended configuration (community area, 4h, log1p, RBF).

2.2 Neural Networks

3 Model Comparison and Recommendation

Both models are evaluated on identical held-out test sets across four configurations, shown in Table 2. As described in the SVM section, the SVR had to be fitted on subsamples of 20,000 and 50,000 rows, while the network trains on the full data of 1.1 million rows for community areas at 1h and 6.8 million for census tracts. Part of the gap between the two models therefore

[2] Due to different scales across temporal and spatial resolutions, the normalized MAE (MAE divided by mean demand) is used to enable a valid comparison.

reflects how much data each could use. We treat this as a property of the methods. Being able to use a dataset of this size is one of the advantages of the neural network as discussed in the lecture.

Table 2: Comparison of SVR and FFNN across matched spatio-temporal configurations

Configuration	SVR R^2	FFNN R^2	SVR rMAE	FFNN rMAE
Community area, 1h	0.83	0.92	0.44	0.20
Census tract, 1h	0.66	0.85	1.17	0.23
Community area, 4h	0.89	0.93	0.32	0.17
Community area, 1d	0.93	0.93	0.23	0.15

On R^2 the two models perform similarly as the time window widens. At 1d both reach 0.93. The difference in performance can be seen with more detailed resolutions. At 1h bins in community areas the SVR reaches 0.83 against 0.92, and on census tracts it falls to 0.66 while the network holds 0.85. The SVR only manages to keep up with the network when the baskets are aggregated into larger ones. The network stays more stable even in finer resolutions. The normalized MAE points in the same direction: the network shows a lower error in every configuration, including the daily setting where R^2 is tied (0.15 against 0.23).

Neither model is simpler to build than the other. The work just sits in different places. The SVR needed target encoding, a log1p transform and a custom scorer to keep kernel selection honest after back-transformation. The network avoids all three, since embeddings handle the spatial units and the target can be used as is, but has to be tuned on architecture depth, learning rate, dropout and early stopping instead. What actually separates them is scale: the SVR cannot use the full dataset, and that shows exactly at the fine resolutions where the extra data matters most.

A more direct test is whether either model beats an operator working without one. We assume a fleet operator would rely on historical averages per community area, 4h basket and weekday/weekend split, essentially saying “on a Tuesday between 8 and 12 in community area 10 we expect 8.2 trips”. This rule of thumb reaches an rMAE of 0.31 on the test set. The SVR does not beat it (0.32), most likely because it only sees a fraction of the training data, while the network reaches 0.17, an improvement of roughly 45 percent. For the SVR the modelling effort does not pay off against a simple historical average; for the network it clearly does. The network is therefore the model we would put into production.

That leaves the question of which resolution to run it at. We recommend community areas in four-hour windows (R^2 0.93, rMAE 0.17). Daily forecasts score marginally better (R^2 0.93, rMAE 0.15) but are too coarse for a fleet operator to work with, while hourly forecasts add noise without adding planning value. Census tracts were considered for their finer spatial detail but rejected. Fewer than half of the records contain census tract information (see Chapter 1), and many tracts generate fewer than one trip per window, which is too granular to allocate vehicles against. Community areas are small enough to act on. The average community area in Chicago covers roughly three square miles,[3] which a taxi can cross in under fifteen minutes.

[3] Chicago comprises 77 community areas across roughly 227 square miles (U.S. Census Bureau).

4 Smart Charging Using Reinforcement Learning

5 Discussion & Outlook

6 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

7 Conclusion

The conclusion should summarize your findings and suggest future work.

References

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.